

What Is Velora?

VELORA is a premium fashion e-commerce platform built from scratch — not a template or boilerplate. It is a **production-grade**, multi-platform retail system for the Colombian market, selling clothing, shoes, hoodies, and accessories with a luxury brand experience. The entire codebase is written in **TypeScript** across a Turborepo monorepo containing three apps (web, mobile, API) and four shared packages.

Tech Stack

FRONTEND

Next.js 15 (App Router) · React 19 · Tailwind CSS 3 · Framer Motion · TanStack React Query · Zustand · Radix UI · Lucide

Next.js · React 19 · Tailwind · React Query · Zustand · Framer

BACKEND & INFRASTRUCTURE

Fastify REST API · Prisma ORM · PostgreSQL · Supabase (Auth + Realtime) · Cloudinary · Wompi Payments

Fastify · Prisma · Supabase · Wompi · Cloudinary

MOBILE & DEVOPS

React Native (Expo) · NativeWind · Docker Compose · Vercel + Render · TypeScript Strict

React Native · NativeWind · Turborepo · Vercel · Render

Architecture

Web (Next.js 15 + React 19)

Tailwind CSS · Framer Motion · Zustand · React Query

API Proxy + Supabase

Fastify API

Prisma · PostgreSQL

Supabase Auth · Realtime

Mobile

Expo · React Native

NativeWind

Wompi

PSE · Nequi · Bre-B · Cards

Cloudinary

Auto format/quality transforms

Key Features

CUSTOMER

- Product catalog w/ filters
- Debounced real-time search
- Animated cart sidebar
- Wompi checkout (4 methods)
- Order tracking timeline
- Bilingual ES/EN

ADMIN

- Product CRUD (inline edit)
- Cloudinary image upload
- Order + shipping management
- Inventory alerts
- Bulk product import

Code Highlights

ZUSTAND CART STORE

```
export const useCartStore = create<CartStore>()(
  persist(
    (set, get) => ({
      items: [] as CartItem[],
      addItem: (product, qty) =>
        set((s) => {
          const existing = s.items.find(
            (i) => i.id === product.id
          )
          return existing
            ? {
                items: s.items.map((i) =>
                  i.id === product.id
                    ? { ...i, quantity: i.quantity + qty }
                    : i
                )
              : { items: [...s.items, { ...product, quantity: qty }] }
        }),
      getTotal: () =>
        get().items.reduce(
          (sum, i) => sum + i.price * i.quantity, 0
        ),
      getItemCount: () =>
        get().items.reduce((sum, i) => sum + i.quantity, 0),
    }),
    { name: 'velora-cart' }
  )
);
```

WOMPI PAYMENT SIGNATURE

CUSTOM I18N HOOK

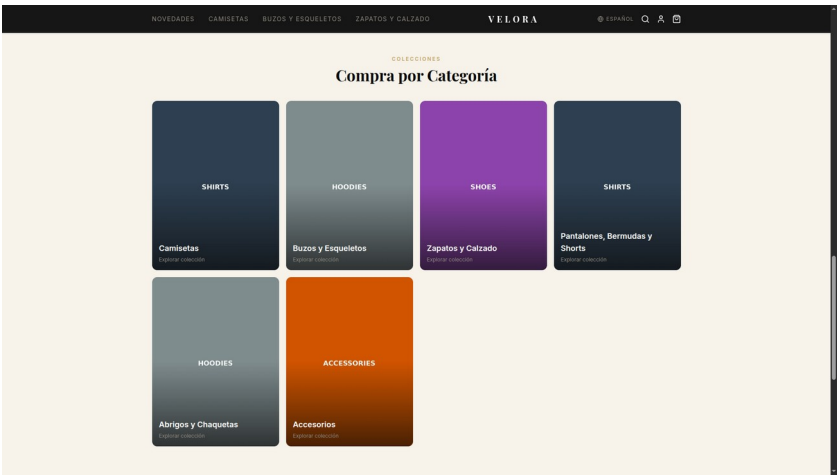
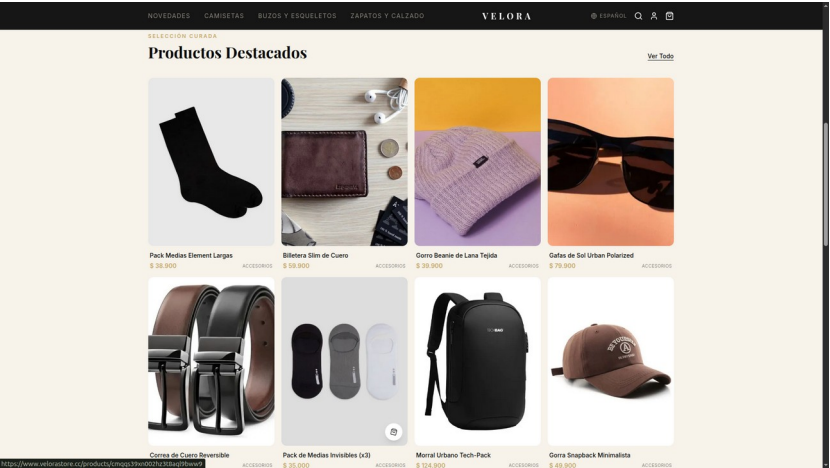
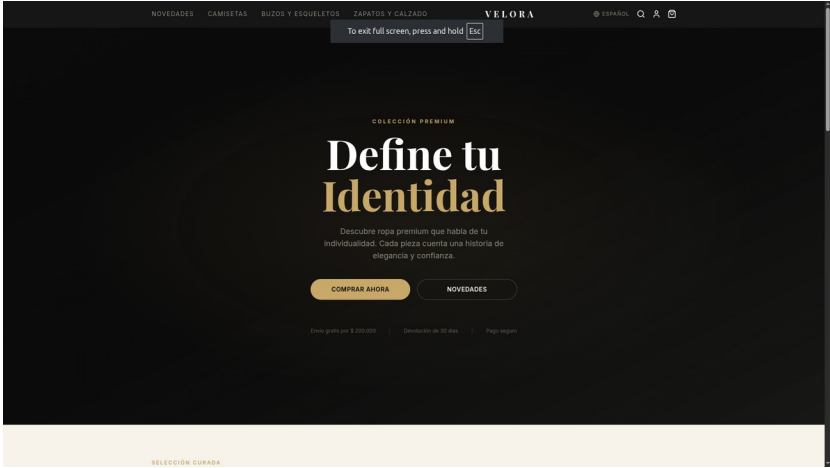
```
export function useTranslation() {
  const { locale } = useLocale();
  const dict = locale === 'es' ? es : en;

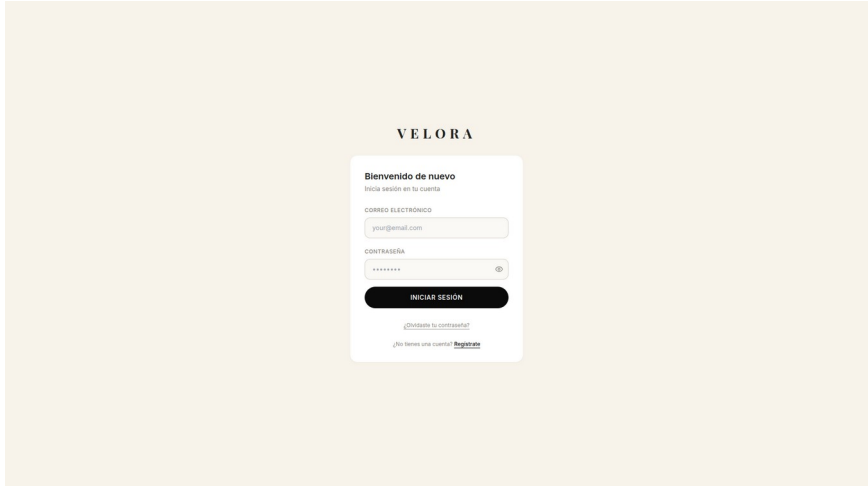
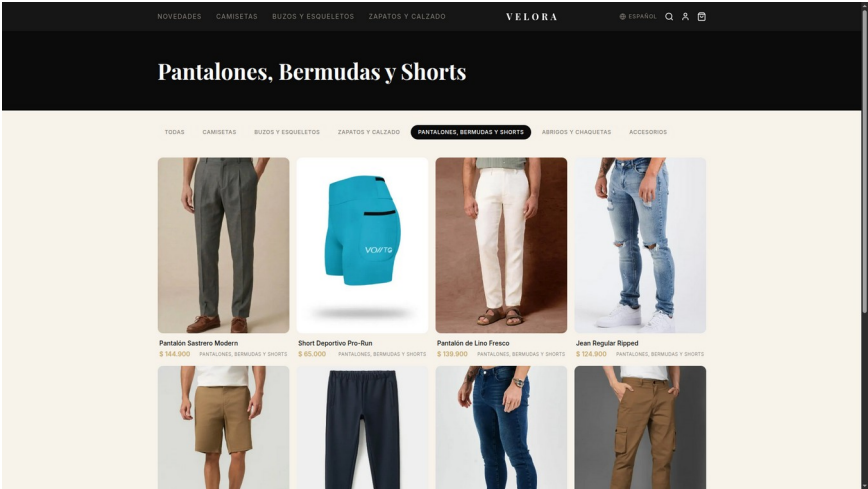
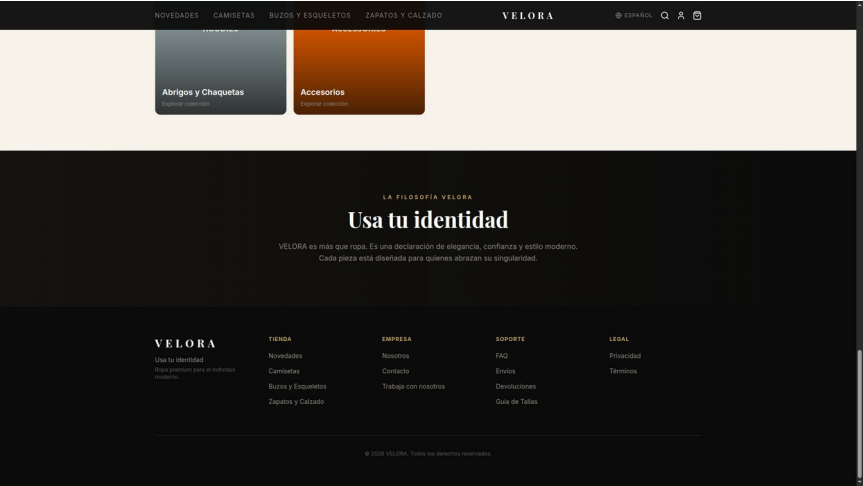
  const t = (path: string,
    params?: Record<string, string>) => {
    const value = path
      .split('.')
      .reduce((obj, key) => obj?.[key], dict);
    if (!params || !value) return value;
    return value.replace(
      /\{(\w+)\}/g,
      (_, k) => params[k] ?? `_${k}`
    );
  };
  return { t, locale };
}
```

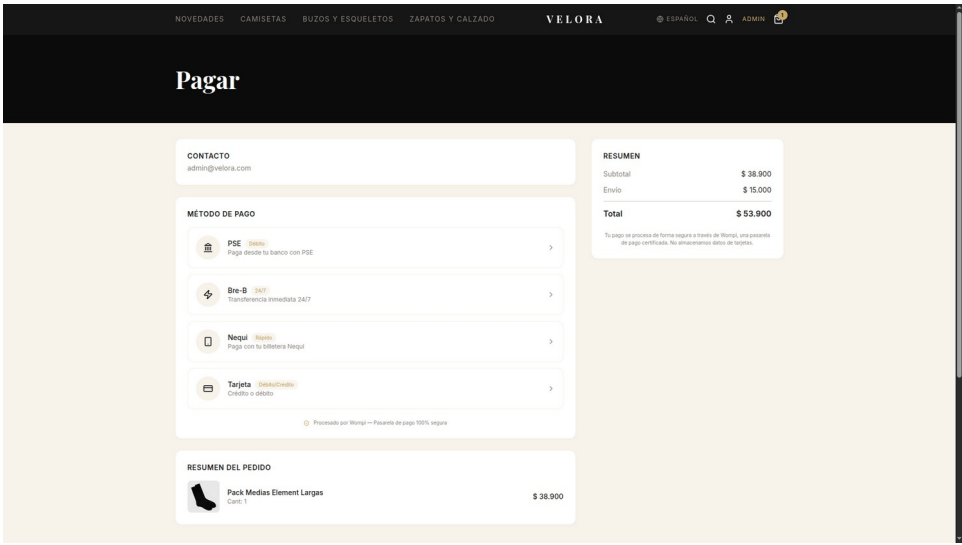
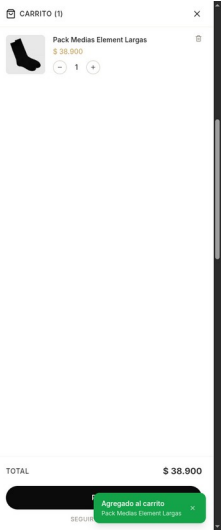
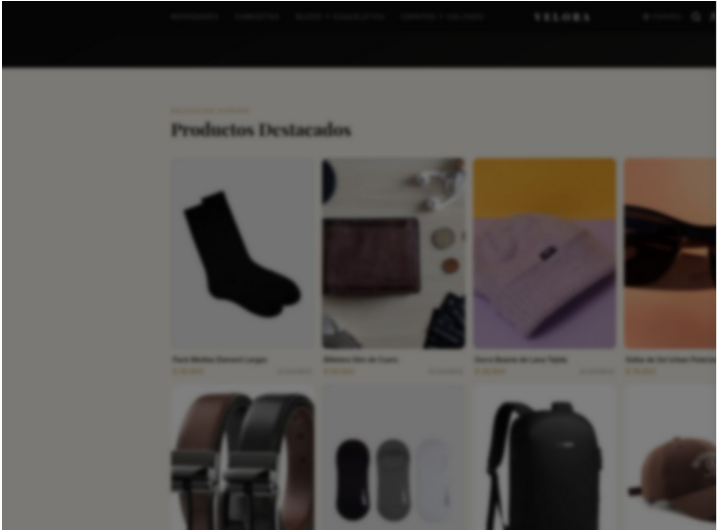
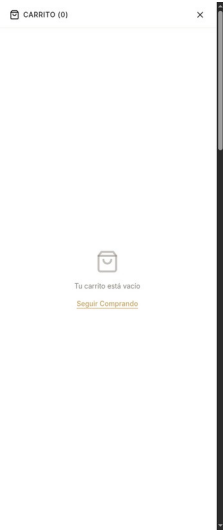
```
const integrity =
  createHmac('sha256', integrityKey)
    .update(`${reference}${amount}COP`)
    .digest('hex');

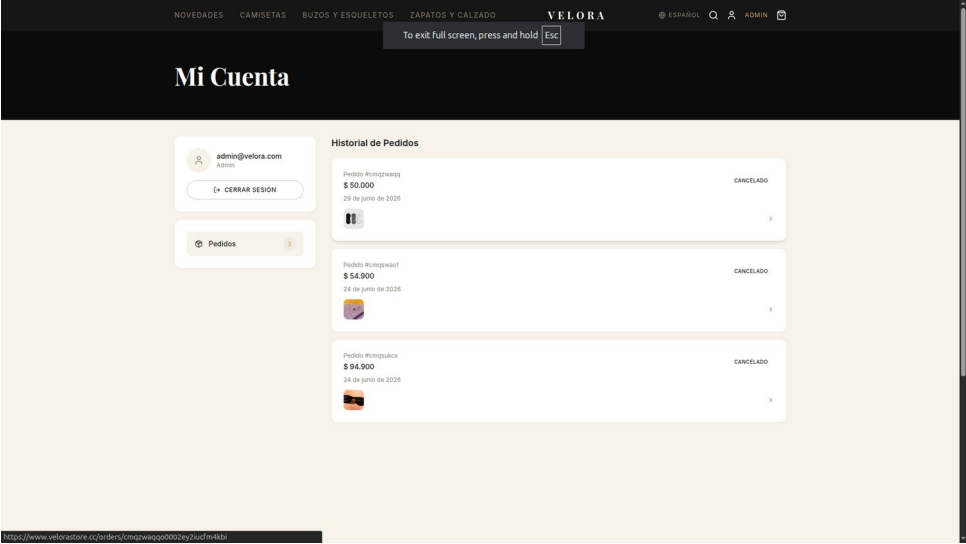
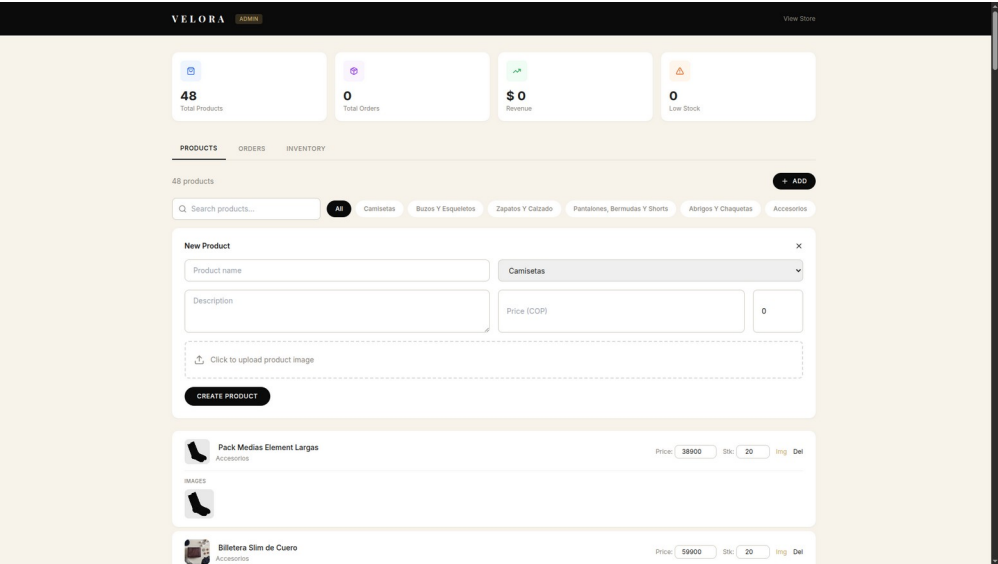
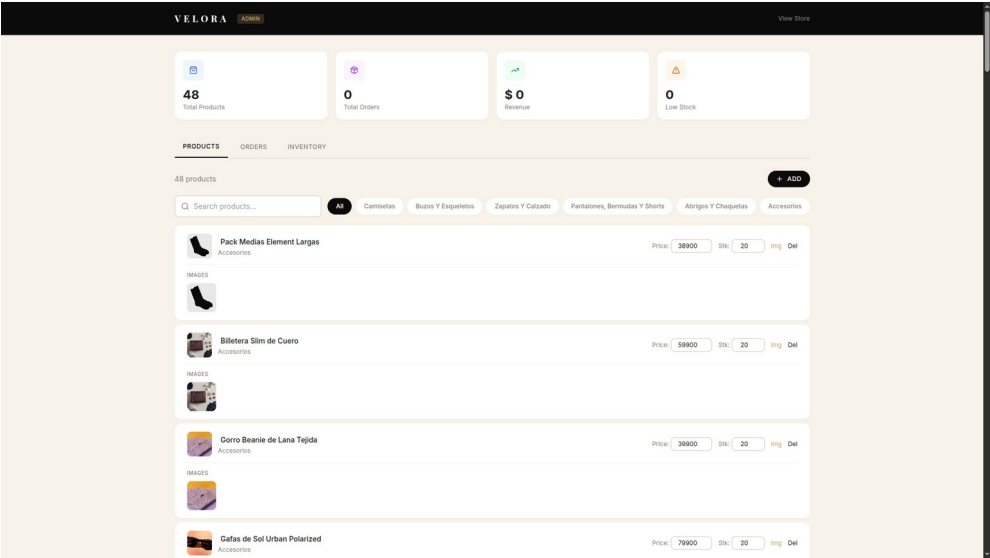
const widget = new Widget({
  publicKey, currency: 'COP',
```

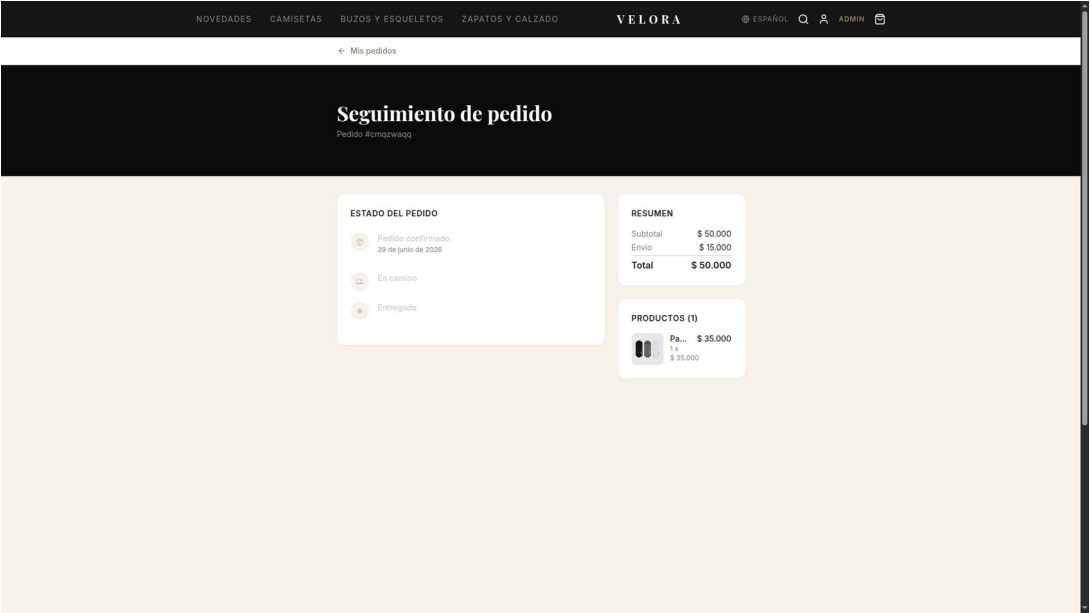
Screenshots











```
export function AuthProvider({
  children }) {
  const { setSession,
    clearSession } =
    useAuthStore();
```

```
useEffect(() =>
{ supabase.auth.getSession().then(({ data }) =>
{
  if (data?.session) setSession(data.session);
});
```